Topic 9 - defining functions

Programming in Science

420-SN1-RE Course Team

Marianopolis College

September 27, 2026

What are functions?

- Programs involve many repeated operations.
- It is useful to *decompose* a large program into smaller pieces.
- *Functions* (or *procedures*) are basic units of program construction.
- A function is *called* by another program or function. When a function completes, it *returns* a value to the *caller*.
- A function is a way to give a name to a series of statements

The def statement

- The Python `def` statement has the form:

```
def name (parameters): # The def header
  statement1           # Statement list
  statement2
  statement3
  ...
```

- The statement list is the function *body*.
- The name will be used to *call* the function.
- The parameters are names of values passed to the function when it is called.

The `return` statement

- A `return` statement does two things:
  - It defines the value of a function call.
  - It returns the flow of control to the caller.
- The syntax is:

  `return` *expression*
- The *optional* expression can have *any* type.
- The entire `return` statement is optional. If not present:
  - The value of the call is `None`.
  - The flow of control returns to the caller at the end of the function body.
- A `return` cannot appear outside a `def` statement.

A first example

```
def square(x):
    """Return x squared."""
    return x ** 2
```

- `x` is the parameter, which holds the value passed to the function.
- An optional *docstring*, typically in triple quotes, describes the function.
- The `return` statement sets the value of the *call*. For example, the after executing the line

```
y = square(2)
```

the value in `y` will be 4.

Calling the square function

```
a = 2
print(square(a))      # Will print 4
print(square(a+2))    # Will print 16
print(square(7.0))    # Will print 49.0
```

- The expression `square(a+2)` calls the function:
  - The expression a+2 is evaluated.
  - The value 4 is assigned to parameter `x`, which is *local* to the function.
  - The flow of control "jumps" to the function body.
  - The value of the function call is `x ** 2`

A second example

```
def gcd(a, b):
    '''Compute the greatest common divisor
    of positive integers 'a' and 'b'.'''
    while a != b:
        if a > b:
            a = a - b
        else:
            b = b - a
    return b # Return value.
```

- The parameters are named `a` and `b`.
- The *docstring* describes the function.
- The body determines the GCD of `a` and `b`.
- The `return` statement sets the value of the *call*.

Calling the gcd function

```
print(gcd(36, 24)) # Will print 12
print(gcd(42, 35)) # Will print 7
print(gcd(7, 36))  # Will print 1
```

- The expression `gcd(36, 24)` calls the function:
  - The two values 36 and 24 are assigned to the names `a` and `b` by position.
  - Th parameter names are *local* to the function.
  - The control flow jumps to the function body.
  - The value remaining in `b` determines the value of the call.
  - The control flow returns to the caller when the `return` runs.

Building functions with functions

```
from gcd import gcd

def lcm(x, y):
    '''Compute the smallest integer
    divisible by both 'x' and 'y'.'''
    return (x * y) // gcd(x, y)
```

- The parameters happen to be named `x` and `y`.

```
print(lcm(14, 21)) # Prints 42
print(lcm(24, 36)) # Prints 72
print(lcm(9, 12))  # Prints 36
```

The importance of local names

- Local names `n`, `r`, and `i` do not affect names assigned outside the function.

```
def factorial(n):
    '''Return n! for positive int 'n'.'''
    r = 1
    for i in range(1, n + 1):
        r = r * i
    return r

r = 0
i = 5
n = factorial(r)
m = factorial(i)
print(n, m, r, i) # prints 1 120 0 5
```

Multiple return statements

```
def iscomposite(m):
    '''Return True if 'm' is composite.'''
    for i in range(2, int(m ** 0.5) + 1):
        if m % i == 0:
            return True
    return False

print(iscomposite(47)) # False
print(iscomposite(49)) # True
```

- A `return` statement exits both the loop and the function *immediately*.
- Will return `True` as soon as a factor is found.

Summary

- Functions are a convenient way to decompose a program into smaller units.
- The `def` statement defines a function.
- A `return` statement ends the function call and sets its value.
- Functions must be defined before we use them.
- Functions parameters and local variables are private to the function.